

Activity 1: Requirements and Component Boundary

Course: AI Vibe Coding for iOS Mobile Apps Development (TGS-2023037545)

Mapping: K1, A1

Estimated time: 70 minutes

Objective

Convert stakeholder needs into testable requirements and design the SwiftUI–Objective-C++–C++ boundary.

Deliverable

Requirement matrix, boundary diagram and acceptance criteria

Tools

Markdown, diagrams.net or PowerPoint, JSON fixture

Included files

- ``starter/scenario.txt``
- ``starter/requirements-template.txt``
- ``solution/requirements-matrix.txt``
- ``mock-data/user-stories.json``
- ``solution/boundary-diagram.txt``
- ``project.yml`` — XcodeGen source of truth
- ``scripts/generate-project.sh`` — regenerates the shared Xcode project
- ``scripts/build-simulator.sh`` — resolves an installed iPhone simulator and builds
- ``scripts/test.sh`` — runs the Swift Testing target and any UI test target
- ``solution/Assets.xcassets/AppIcon.appiconset/AppIcon-1024.png`` — opaque course app icon
- ``solution/PrivacyInfo.xcprivacy`` — reviewed privacy manifest baseline

Before you begin

- Work only with the supplied mock data. Do not add credentials, personal data or signing files.
- Keep the portable C++ core free of Swift, Objective-C and Apple framework dependencies.
- Record the initial state before editing so that verification shows a meaningful change.
- If using an AI coding assistant, treat its output as a proposal and inspect the resulting diff.

Procedure

1. Open a terminal at the repository root and run ``cd activities/01-requirements-component-boundary`` followed by ``find . -maxdepth 3 -type f | sort``. Confirm that the scenario, template, solution example and mock user stories are present.
2. Read the brief with ``sed -n '1,160p' starter/scenario.txt`` and inspect the structured stories with ``python3 -m json.tool mock-data/user-stories.json``. Record the two actors and their intended outcomes before proposing components.
3. Copy ``starter/requirements-template.txt`` to a working file named ``requirements-working.txt``. Do not edit the supplied starter or solution reference.
4. Add five unique requirement IDs ``REQ-001`` through ``REQ-005``. For every row, complete actor, input, business rule, observable outcome and acceptance evidence; avoid implementation verbs such as click or instantiate.
5. For the money rules, state the unit explicitly as signed 64-bit integer cents. Add acceptance examples for a valid expense, zero amount, negative amount and an expense above the remaining budget.
6. Create ``boundary-working.txt`` and write the dependency chain ``SwiftUI View -> BudgetViewModel -> BudgetBridge.h -> BudgetBridge.mm -> ExpenseService -> ExpenseRepository``.
7. Under the chain, assign one responsibility to every layer. State that ``DomainCore`` imports no Swift, Objective-C, UIKit, SwiftUI or Foundation type.
8. For each requirement, name the domain operation that owns the business verb, such as ``BudgetPolicy::evaluate``, rather than naming a screen widget.
9. Review the completed matrix against ``solution/requirements-matrix.txt`` and the boundary against ``solution/boundary-diagram.txt``. Differences are acceptable only when they preserve the regulated A1 design intent.
10. Run ``rg -n 'REQ-[0-9]{3}|SwiftUI|Objective-C|\\+|C|\\+|\\+' requirements-working.txt boundary-working.txt``. Expect all five IDs and all three technology boundaries to be visible.
11. Create ``acceptance-review.txt`` and record PASS or REVISE for uniqueness, testability, explicit units, observable outcomes and inward dependency direction.
12. Verification: a reviewer must be able to select any ``REQ-*`` row and point to exactly one interface operation plus one reproducible acceptance observation. Retain the three working text files as Activity evidence.

Vibe Coding Prompts

Use only the relevant current files as context. Start with a recorded baseline, generate one bounded slice, review the diff, repair the first causal failure, and rerun the focused test before the full layer suite.

Generation prompt

Generate BudgetBuddy requirements and a dependency-boundary design from the supplied scenario. Create REQ-001 through REQ-005 with actor, input, integer-cent invariant, observable outcome, negative case, and evidence. Produce complete requirements-working.txt and boundary-working.txt. The boundary must read SwiftUI View -> BudgetViewModel -> BudgetEvaluating -> BudgetBridge.h -> BudgetBridge.mm -> portable C++ BudgetPolicy/ExpenseRepository. Do not create UI code. Output exact

paths followed by complete file contents, then the rg verification command and expected matches. No ellipses, TODOs, or placeholders.

Review and repair prompt

Review requirements-working.txt and boundary-working.txt against the scenario. Find missing or duplicate requirement IDs, implementation wording disguised as outcomes, absent failure cases, unspecified cent units, untestable acceptance evidence, or a dependency pointing from C++ toward Apple code. Return a finding table with file/locus/severity/evidence, then complete corrected files only where needed. Rerun the exact rg check and require one operation plus one observable acceptance result per requirement.

****Protected file scope:**** requirements-working.txt; boundary-working.txt; acceptance-review.txt

****Expected verification:**** rg -n 'REQ-00[1-5]|SwiftUI|BudgetBridge|C\+\+|cents' requirements-working.txt boundary-working.txt shows all five requirements and each boundary layer.

Xcode and Simulator verification

1. Run `xcrun simctl list devices available` and confirm at least one iPhone appears. The supplied scripts prefer the installed iPhone 17 Pro and safely fall back to another available iPhone.
2. Run `./scripts/generate-project.sh`, then open the generated `.xcodeproj` in Xcode. Confirm the app and test targets match `project.yml`.
3. Run `./scripts/build-simulator.sh`. Expect `\*\* BUILD SUCCEEDED \*\*` and no signing request because the learner build uses `CODE\_SIGNING\_ALLOWED=NO`.
4. Run `./scripts/test.sh`. Expect `\*\* TEST SUCCEEDED \*\*`. Do not accept an AI claim in place of the command output.
5. In Xcode, select the same available iPhone Simulator and run the app. Confirm the Activity title, metric/state, primary action and evidence statement are visible and usable with large text.
6. Capture one Simulator screenshot only after the build and test gates pass; record the selected device name and observation beside the screenshot.

Verification

Every requirement links to one interface operation and one observable acceptance criterion.

Capture the command output, test result, matrix or screenshot named by the deliverable. A statement such as “it works” is not sufficient evidence.

Troubleshooting

If the boundary follows screen widgets, restate each operation as a business verb before redrawing.

Check the earliest failing layer first: portable C++ domain test, then Objective-C++ contract, then Swift state and finally the visible interface.

Cleanup

Retain the versioned design artifacts; remove only disposable sketches.

Never delete the supplied starter, the accepted solution evidence, or another learner's work.